This CVPR paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

LPOSS: Label Propagation Over Patches and Pixels for Open-vocabulary Semantic Segmentation

Vladan Stojnić[1] Yannis Kalantidis[2] Jiří Matas[1] Giorgos Tolias[1]

[1] VRG, FEE, Czech Technical University in Prague [2] NAVER LABS Europe

Abstract

We propose a training-free method for open-vocabulary semantic segmentation using Vision-and-Language Models (VLMs). Our approach enhances the initial per-patch predictions of VLMs through label propagation, which jointly optimizes predictions by incorporating patch-to-patch relationships. Since VLMs are primarily optimized for cross-modal alignment and not for intra-modal similarity, we use a Vision Model (VM) that is observed to better capture these relationships. We address resolution limitations inherent to patch-based encoders by applying label propagation at the pixel level as a refinement step, significantly improving segmentation accuracy near class boundaries. Our method, called LPOSS+, performs inference over the entire image, avoiding window-based processing and thereby capturing contextual interactions across the full image. LPOSS+ achieves state-of-the-art performance among training-free methods, across a diverse set of datasets. Code: https://github.com/vladan-stojnic/LPOSS

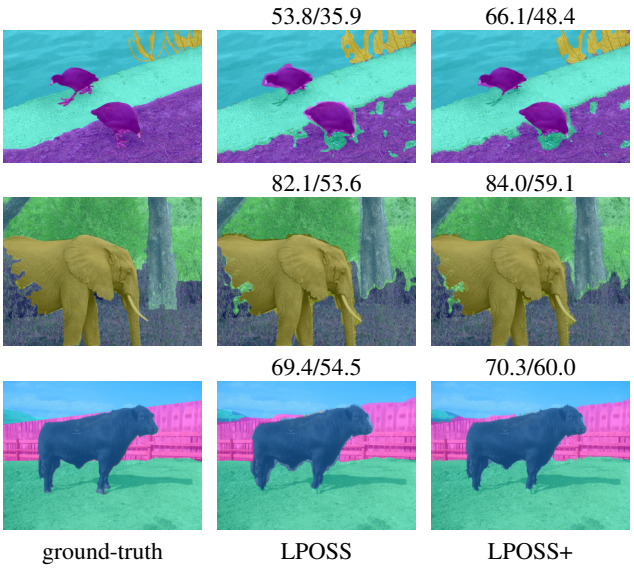


Figure 1. Open-vocabulary semantic segmentation with LPOSS (patch-level LP) and LPOSS+ (pixel-level LP). Performance reported via mIoU / Boundary IoU. Images from the Context and COCO-Stuff datasets.

1. Introduction

Semantic segmentation models are typically trained in a supervised manner, requiring pixel-level annotations based on a predefined set of categories or classes. These approaches face significant scalability challenges due to the high cost of annotation, and their applicability is limited by the fixed set of classes they are trained to recognize. Open-vocabulary semantic segmentation aims to generalize and allow the segmentation of an image into any given list of classes provided at inference time. This task is usually solved with the use of a Vision-and-Language Model (VLM) [10, 22, 33].

VLMs produce aligned textual and visual features and enable matching textual descriptions, such as class names, to visual features. While highly effective for image-level classification tasks, VLMs are less suited for dense prediction tasks like semantic segmentation out-of-the-box, as they are trained to align global image representations with textual data. Our work addresses this gap by proposing a novel, training-free VLM enhancement for open-vocabulary semantic segmentation.

Although segmentation is defined as pixel-wise labeling, recent approaches, that rely on VLMs [26, 42, 46] utilizing ViT [15] architecture, form predictions at the patch level and simply up-sample the predictions. A ViT-based VLM typically provides a class probability distribution independently for each patch, yielding an initial prediction. Our approach incorporates the initial prediction together with patch relationships by making joint predictions across all patches. This is achieved through minimizing a quadratic cost function with unary and binary terms. This approach ensures that final predictions remain close to initial ones while nearby patches or patches with similar appearances produce similar predictions, reflecting a common prior in semantic segmentation. Optimization is performed through

either an iterative Label Propagation (LP) [54] operation on a patch graph or, equivalently, by solving a linear system. The latter is computationally efficient due to the high sparsity of the coefficient matrix and the use of GPU-based solvers. Following prior work [23, 26, 46], and due to the fact that VLMs optimize inter-modal rather than intra-modal similarity, we leverage a strong Vision Model (VM), specifically DINO [5], to effectively capture patch-to-patch appearance and relationships. Using such an encoder, instead of the VLM's visual encoder, significantly improves predictions at a patch level.

While operating at the patch level provides efficiency benefits, it constrains pixel-level accuracy. In an experiment where semantic segmentation ground truth is downsampled to patch-level resolution, then upsampled and evaluated as a prediction, we show that the resulting performance measured via mIoU averages approximately 85% across eight datasets. To address this limitation, we perform a second stage of LP on a pixel-level graph, using the patch-based LP result as the initial prediction set. This refinement consistently improves segmentation accuracy, particularly near class boundaries, as we show by measuring the Boundary IoU metric [9]. Figure 1 demonstrates examples of such improvement using VLM and VM with a ViT backbone.

The strength of ViT architectures comes with limitations. There is a strong bias introduced by their pre-training settings, *i.e*. processing fixed-resolution square images. Consequently, current segmentation methods perform feature extraction and prediction in a sliding window fashion with overlapping windows. In contrast, we move beyond this standard practice by performing predictions jointly for the entire image, accounting for interactions across all patches. However, we show that while joint predictions are possible, joint feature extraction across the whole image is not feasible without a noticeable performance decrease. Our proposed method, called *Label Propagation over Patches* (LPOSS) *and Pixels* (LPOSS+), achieves state-of-the-art results across various datasets, outperforming existing training-free methods on eight datasets. Our contributions are summarized as follows:

- We establish a connection between classical label propagation methods and vision-language models (VLMs) in the context of dense prediction tasks.
- We introduce a training-free approach for semantic segmentation that predicts and propagates labels at both patch and pixel levels. This strategy leverages the pre-training of foundation models and addresses the performance limitations of coarse patch-level predictions.
- We propose to decouple feature extraction from class prediction, enabling image-level inference and capturing contextual interactions across the entire image.

2. Related work

The introduction of vision-language models [10, 22, 33] opened up the possibilities of performing many recognition tasks in the open-vocabulary setting [43], *e.g*., zero-shot classification [33, 38, 41], semantic segmentation [17, 53], object detection [44], *etc*. However, considering these models are usually trained only to optimize global image representation, they do not excel in dense recognition tasks such as segmentation [3, 53]. Because of this, considerable work is being done to improve their performance. This is achieved by training specialized VLM for such tasks, changing the VLM architecture during inference, and employing other vision models to aid VLMs.

Training VLMs for segmentation. One line of work trains from scratch VLMs specialized for semantic segmentation using varying levels of supervision: using segmentation masks without class annotations [14, 17] and only image-caption pairs [29, 34, 47, 48]. On the other hand, another line of work trains additional modules on top of pre-trained VLM to improve their localization abilities [6, 28, 31].

Training-free segmentation with VLMs. It is observed [27, 42] that different VLM components are responsible for VLMs bad localization performance and that the performance can be substantially improved by removing or changing them during inference. MaskCLIP [53] revisits the global attention pooling of CLIP [33] and proposes to remove its query and key embeddings while reformulating the value embedding layer and the last linear layer into two 1×1 convolutions. SCLIP [42] proposes to replace the self-attention layer of the ViT with correlative self-attention. ClearCLIP [25] identifies residual connections as the primary source of VLMs' bad localization performance. Because of this, it proposes to remove them from the last layer. It additionally replaces self-attention with self-self-attention and removes the feed-forward network from the last layer. GEM [3] improves the semantic segmentation abilities of VLMs by adding self-self-attention blocks parallel to self-attention layers. We consider these methods complementary to ours as we apply our method on top of MaskCLIP [53].

VLMs and VMs for segmentation. Vision models trained in a self-supervised fashion show remarkable object localization abilities [5, 13, 32, 36, 37]. Because of this, the growing line of work investigates how to leverage them to improve the ability of VLMs to perform open-vocabulary semantic segmentation. CLIP-DIY [45] uses an unsupervised object discovery method FOUND [37] to clear and improve the open-vocabulary segmentation maps of CLIP [33]. CLIP-DINOiser [46] proposes a training-free variant that uses patch-to-patch affinity from DINO [5] to refine CLIP's image features. Additionally, it shows that good localization capabilities can be extracted from CLIP by training two small convolutional layers. ProxyCLIP [26]

proposes replacing the attention matrix of the last layer of CLIP [33] with an affinity matrix from a vision model such as DINO [5]. LaVG performs panoptic segmentation using the normalized cut on top of DINO [5] features. The discovered segments are then classified into a class using a VLM. Our method falls into this group of methods and is most similar to CLIP-DINOiser [46]. Compared to CLIP-DINOiser, which uses affinity based on Euclidean similarities between DINO patches, we propose to use geodesic similarities obtained through label propagation. Additionally, compared to all other works, we perform the prediction jointly across all sliding windows, which are usually used in such methods. Finally, we depart from performing predictions only at the patch level and refine the predictions at the pixel level by applying the same process on top of pixels.

Relation to CRFs. Conditional Random Fields (CRFs) [24, 40] have a long tradition in dense prediction tasks with classical approaches [18, 24, 35]. In early deep models for semantic segmentation, CRFs are used as a refinement step to increase the accuracy and the resolution of the predictions [8]. There are also attempts to integrate a similar objective function during end-to-end training [7, 50]. CRFs and LP both leverage pairwise connections to achieve smooth label assignments. LP directly propagates labels based on neighbors, while CRFs explicitly model these relationships in their energy function. We opt for an LP variant that has theoretical convergence proofs and corresponds to an intuitive quadratic cost function, which does not hold for all existing LP variants [2]. LP is typically easy to implement, computationally efficient [54], and compatible with GPU-based software implementations [38]. Consequently, it is used in various computer vision and machine learning tasks [19, 21, 38, 39], while in this work, LP is tailored for semantic segmentation in the era of VLMs and ViT encoders.

3. Label propagation over patches and pixels

3.1. Problem formulation

In the task of open-vocabulary semantic segmentation we are given an image $X \in \mathbb{R}^{H\times W\times 3}$ and a list of class names $\mathcal{C} = \{c_1, \dots, c_C\}$. Our goal is to assign one of the classes to every pixel or, conversely, tensor $Y \in \mathbb{R}^{H\times W\times C}$ that contains a segmentation mask for each of the C classes.

To solve such a task, one can utilize a Vision Language Model (VLM), *i.e.* a model that is able to encode both the class names as well as the image in the compatible feature spaces. Let $f : \mathcal{I} \to \mathbb{R}^{d\times N_P}$ denote the vision encoder, *i.e.* a ViT [15] with patch size P. The vision encoder extracts feature vectors for all $N_P = \lceil H/P \rceil \cdot \lceil W/P \rceil$ patches for an image in $\mathcal{I}$[1]. Also, let $g : \mathcal{T} \to \mathbb{R}^d$ represent the text encoder of the VLM, with $\mathcal{T}$ representing the space of the textual input. One can, therefore, perform open-vocabulary semantic segmentation by obtaining the patch features for an image $Z_{\text{vlm}} = f(X)$, class name features $F = g(\mathcal{C}) \in \mathbb{R}^{d\times C}$, and calculating per patch class similarities

$$Y_{\text{vlm}} = Z_{\text{vlm}}^{\top} F, \tag{1}$$

which are further upsampled to pixel-wise class similarities at the original image size. The class with the highest similarity is then assigned to each pixel.

3.2. Preliminaries

Label propagation (LP). Let $\{n_1, \dots, n_N\}$ be a set of graph nodes and $S \in \mathbb{R}^{N\times N}$ be a symmetric, typically very sparse, adjacency matrix with zero diagonal. We obtain a symmetrically normalized adjacency matrix by $\hat{S} = D^{-\frac{1}{2}} S D^{-\frac{1}{2}}$, where $D = \text{diag}(S\mathbf{1}_N)$ is a degree matrix and $\mathbf{1}_N$ is the all-ones N-dimensional vector. Given $\hat{S}$, label propagation [54] is an iterative process given by

$$\hat{Y}^{(t+1)} = \alpha \hat{S} \hat{Y}^{(t)} + (1-\alpha) Y \tag{2}$$

until convergence, that refines initial node predictions $Y \in \mathbb{R}^{N\times C}$ across C classes based on the graph structure[2]. Hyper-parameter $\alpha \in (0, 1)$ controls the amount of propagation and refinement. It is possible to show [54] that this iterative solution is equivalent to solving C N-variable linear systems

$$\mathcal{L}(S)\hat{Y} = Y, \tag{3}$$

where $\mathcal{L}(S) = I - \alpha D^{-\frac{1}{2}} S D^{-\frac{1}{2}} = I - \alpha \hat{S}$ is the graph Laplacian. The closed-form solution of these linear systems

$$\hat{Y} = \mathcal{L}(S)^{-1} Y = L_{\text{inv}}(S) Y, \tag{4}$$

is not practical for large graphs as $L_{\text{inv}}(S)$ is a dense $\mathbb{R}^{N\times N}$ matrix. However, given that L is positive-definite, it is usual [20, 21] to solve (3) using the conjugate-gradient method which is known to be faster than running the iterative solution [20]. Matrix L_{inv} provides geodesic similarities between all pairs of nodes, as these are captured based on the graph structure. Additionally, label propagation corresponds to minimizing the quadratic criterion

$$Q(\hat{Y}) = (1-\alpha) \sum_{i=1}^{N} ||\hat{Y}_i - Y_i||^2 + \alpha \sum_{i,j=1}^{N} S_{ij} ||D^{-\frac{1}{2}} \hat{Y}_i - D^{-\frac{1}{2}} \hat{Y}_j||^2. \tag{5}$$

The criterion (5) consists of two parts: the first one keeps node predictions close to the initial ones, and the second one pushes neighboring nodes in the graph to have the same predictions.

[1] We discard the representation corresponding to the CLS token usually present in ViTs.

[2] A common use of LP is to propagate from labeled to unlabeled nodes; instead we refine predictions for all nodes which are already labeled.

Exploiting a Vision Model (VM). VLMs are trained by minimizing cross-modal similarities between corresponding visual and textual features. This training approach results in suboptimal performance on dense prediction tasks, such as open vocabulary semantic segmentation [53], for two main reasons. First, VLMs are typically trained using a loss on *global* image representations, which means they do not explicitly optimize patch-level features. Second, VLM training focuses solely on cross-modal similarities and does not account for relationships across patches, making the resulting representations less effective for capturing the inter-patch relationships essential for semantic segmentation.

For these reasons, recent works [23, 26, 46] further utilize a separate *Vision Model (VM)* to provide improved patch-level representations. Let $Z_{\text{vm}} = h(X)$ denote patch-level features from such a model, where $h : \mathcal{I} \to \mathbb{R}^{d' \times N_P}$ is an encoder with a ViT architecture. The usual choice for the vision model is DINO [5], an encoder trained with self-supervised learning that exhibits strong performance for dense prediction tasks [5, 36, 37].

The training-free variant of the CLIP-DINOiser [46] improves VLM predictions Y_{vlm} by refining VLM image features Z_{vlm} using patch relations from VM. They use image features Z_{vm} and compute an affinity matrix $A = Z_{\text{vm}}^\top Z_{\text{vm}}$. This affinity is then used to refine the VLM predictions as

$$Y_{\text{DINOiser}} = AZ_{\text{vlm}}^\top F = AY_{\text{vlm}}. \tag{6}$$

This process propagates the patch features, or, equivalently, due to linearity, the VLM predictions, over the affinity matrix[3]. This is a smoothening operation, which we also perform in our method but in a more principled way.

Class prediction using a sliding window approach. Current approaches [23, 26, 42, 46] perform feature extraction and prediction in a sliding-window fashion to handle test images of arbitrary resolution. Let $\mathcal{W}$ denote a set of K sliding windows. Prediction typically proceeds in 3 steps:

1. Extract features $Z_{\text{vlm}}^{(w_i)}$ and obtain predictions $Y_{\text{vlm}}^{(w_i)} = {Z_{\text{vlm}}^{(w_i)}}^\top F$ independently for each window $w_i \in \mathcal{W}$.
2. Upsample patch-level predictions to the original window resolution using function $\texttt{up}(Y_{\text{vlm}}^{(w_i)})$.
3. Obtain per-pixel predictions by averaging predictions over all sliding windows that contain each pixel and combine them into an $H \times W$ class confidence map $Y_{\text{vlm}}^{(img)} = \texttt{combine}(\{\texttt{up}(Y_{\text{vlm}}^{(w_1)}), \dots, \texttt{up}(Y_{\text{vlm}}^{(w_K)})\})$.

The sliding window approach presented above is a result of the poor generalization of ViTs to resolutions and aspect ratios different from the ones used during training. All existing approaches couple feature extraction with class prediction and perform both *jointly*, and *independently in each window* before averaging predictions across windows.

[3]Their implementation ℓ_2-normalizes features after propagation, *i.e.* $\ell_2(AX)F^\top$, which breaks the equivalence but performs slightly better.

In the following sections, we decouple the two for our approach and perform predictions by looking across windows.

3.3. LPOSS: LP over patches

We observe that (6) and (4) can both be seen as different forms of label refinement. In (6), the VLM-based labels Y_{vlm} are refined using matrix A of Euclidean similarities across patches, while in (4), graph labels Y are refined via the geodesic similarities L_{inv}, computed over the graph. This motivates us to utilize label propagation to refine the VLM predictions. Not only is (4) optimizing (5), which is a principled and intuitive objective function, but we also expect geodesic similarities to capture more complex relations and contextual information, as they are computed by considering the entire set of patch features together.

We assume a patch $\mathtt{p}_i = (\mathbf{z}_{\text{vm}_i}, \mathbf{p}_i)$ is represented as a tuple by its features $\mathbf{z}_{\text{vm}_i}$ from the VM, *i.e.* the i-th column of Z_{vm}, and the location of its center $\mathbf{p}_i$ in the image. We then construct a graph with nodes $\{\mathtt{p}_1, \dots, \mathtt{p}_{N_P}\}$ and adjacency matrix $S_{\mathtt{P}} = S_a \odot S_p$, where $\odot$ denotes element-wise multiplication. S_a is an appearance-based adjacency matrix and $s_{a_{ij}} = (\mathbf{z}_{\text{vm}_i}^\top \mathbf{z}_{\text{vm}_j})^\gamma$ if $\mathbf{z}_{\text{vm}_j}$ is in the k-nearest neighbors of $\mathbf{z}_{\text{vm}_i}$, and 0 otherwise. S_p is a position-based adjacency matrix that depends on the spatial distance between patches with $s_{p_{ij}} = \exp\left(-\frac{||\mathbf{p}_i - \mathbf{p}_j||^2}{\sigma}\right)$. Having such a graph, we apply label propagation by

$$Y_{\text{lposs}} = L_{\text{inv}}(S_{\mathtt{P}})Y_{\text{vlm}} \tag{7}$$

to refine the VLM predictions. This process effectively refines the predictions such that neighboring patches (patches with high appearance similarity or spatial proximity) have similar predictions, while keeping the final prediction not far from the VLM predictions.

3.4. LPOSS across all windows

We propose to decouple the feature extraction process from class prediction. First, we perform feature extraction on a per-window basis to be consistent with the encoder's training resolution. Unlike existing methods, however, we then carry out predictions *across windows*, *i.e.*, jointly for the entire image. This approach enables us to explicitly account for interactions across all patches.

Sliding-window extraction provides us with baseline VLM predictions $Y_{\text{vlm}}^{(w_i)}$ for window w_i with patches $\mathtt{p}_j^{(w_i)}, j = 1, \dots, N_P$, which we merge in $Y_{\text{vlm}}^{(\mathcal{W})} = [{Y_{\text{vlm}}^{(w_1)}}^\top, \dots, {Y_{\text{vlm}}^{(w_K)}}^\top]^\top$, across all windows. We construct affinity matrix $S^{(\mathcal{W})} \in \mathbb{R}^{KN_P \times KN_P}$ over nodes $\{\mathtt{p}_1^{(w_1)}, \dots, \mathtt{p}_{N_P}^{(w_1)}, \dots, \mathtt{p}_1^{(w_K)}, \dots, \mathtt{p}_{N_P}^{(w_K)}\}$ containing patches from all sliding windows, and perform LP jointly for the whole image by

$$Y_{\text{lposs}}^{(\mathcal{W})} = L_{\text{inv}}(S^{\mathcal{W}})Y_{\text{vlm}}^{(\mathcal{W})}. \tag{8}$$

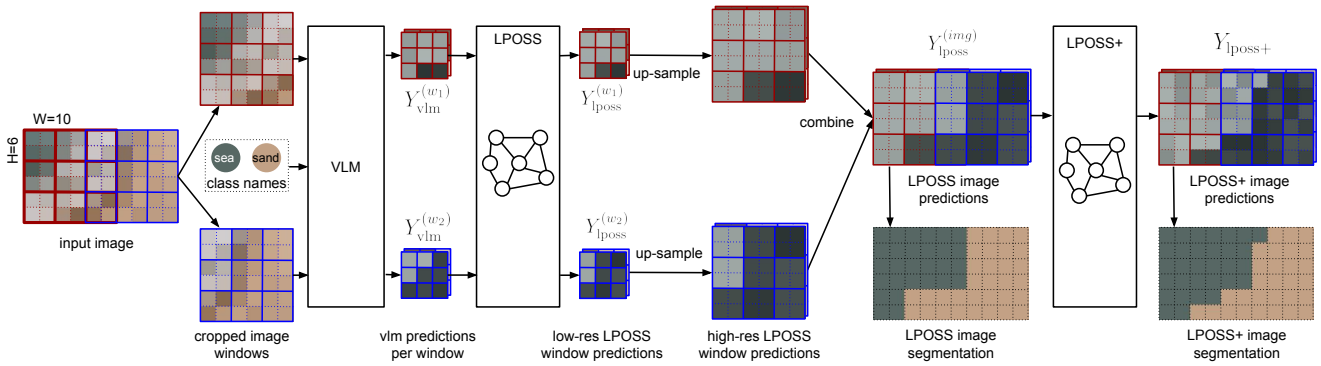


Figure 2. **Overview of LPOSS and LPOSS+.** Processing steps: (1) input image window cropping (2) feature extraction with the VLM and patch-level predictions per window (3) patch-level predictions jointly across all windows with LPOSS (4) upsampling of patch-level predictions (5) combining window predictions into a single pixel-level image prediction (6) pixel-level prediction with LPOSS+. The graphs of LPOSS and LPOSS+ are generated based on patch features extracted with the VM and pixel color values, respectively.

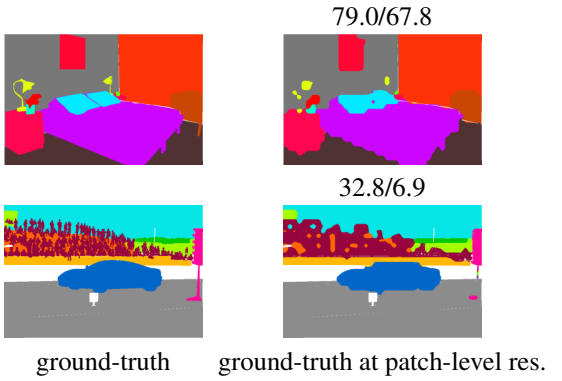


Figure 3. **Visualization of artifacts caused by patch-level predictions.** Ground-truth at patch-level resolution produced by downsampling the ground-truth by the patch size $P = 16$ and then upsampling to the original size. Performance reported via mIoU / Boundary IoU.

We decompose the output prediction into parts associated with each window by $Y_{\text{lposs}}^{(\mathcal{W})} = [{Y_{\text{lposs}}^{(w_1)}}^\top, \ldots, {Y_{\text{lposs}}^{(w_K)}}^\top]^\top$, and, similar to other approaches, upsample and combine by

$$Y_{\text{lposs}}^{(img)} = \texttt{combine}(\{\texttt{up}(Y_{\text{lposs}}^{(w_1)}), \ldots, \texttt{up}(Y_{\text{lposs}}^{(w_K)})\}), \tag{9}$$

with $Y_{\text{lposs}}^{(img)} \in \mathbb{R}^{H \times W \times C}$. We show a graphical representation of this process in Figure 2.

3.5. LPOSS+: LP over pixels

Similar to related approaches, LPOSS performs predictions at patch resolution, which are subsequently upsampled to the original image resolution. However, we observe that this process results in block-like artifacts in the predictions, as shown in Figure 3.

To assess the impact of these artifacts on performance, we conducted the following experiment on the ground-truth maps: we downsampled the ground-truth map to patch-level resolution by a factor corresponding to the patch size (e.g., $P = 16$), then upsampled it back to the original image resolution. We treat the result as a prediction and evaluate performance using mIoU and Boundary IoU [9] relative to the ground truth. The average performance across eight datasets was approximately 85% for mIoU and 70% for Boundary IoU, as shown in Table 1 and Table 2, indicating that patch-level predictions inherently limit performance. To address this, we propose to apply label propagation at the pixel level to further refine LPOSS predictions $Y_{\text{lposs}}^{(img)}$.

We assume a pixel $\tilde{\mathbf{p}}_i = (\tilde{\mathbf{z}}_i, \tilde{\mathbf{p}}_i)$ to be represented as a tuple by its location $\tilde{\mathbf{p}}_i$ within the image and its features $\tilde{\mathbf{z}}_i$, which may be based on the RGB values or any encoder. Similar to the patch-level case, we construct a graph with pixels as nodes and adjacency matrix $S_{\tilde{\mathbf{p}}} = S_{\tilde{a}} \odot S_{\tilde{p}}$. The position-based affinity $S_{\tilde{p}}$ is binary, with non-zero elements within the $r \times r$ neighborhood of a pixel, and the appearance-based affinity $S_{\tilde{a}}$ has elements equal to $s_{\tilde{a}_{ij}} = \exp\left(-\frac{||\tilde{\mathbf{z}}_i - \tilde{\mathbf{z}}_j||^2}{\tau}\right)$.

We apply label propagation on top of the pixel-level graph by

$$Y_{\text{lposs+}} = L_{\text{inv}}(S_{\tilde{\mathbf{p}}}) Y_{\text{lposs}}^{(img)}, \tag{10}$$

with $Y_{\text{lposs+}}^{(img)} \in \mathbb{R}^{H \times W \times C}$, which, after argmax, becomes the final segmentation mask. This process refines the predictions such that neighboring pixels have similar predictions, while keeping the final prediction not far from the LPOSS predictions. We call this approach **LPOSS+** and show its visualization in Figure 2.

4. Experiments

4.1. Datasets and evaluation metrics

We evaluate our method on eight datasets: PASCAL VOC [16] (VOC), COCO Object [4] (Object), PASCAL Context [30] (Context), PASCAL Context59 [30] (C59), COCO Stuff [4] (Stuff), PASCAL VOC20 [16] (VOC20), ADE20k [51, 52], and Cityscapes [12] (City), that are commonly used [23, 26, 42, 46] to evaluate open-vocabulary

Method	VM	VOC	Object	Context	C59	Stuff	VOC20	ADE20k	City	Avg
Oracle (patch-level res.)	-	91.5	83.8	86.6	86.8	86.0	92.8	80.6	73.6	85.2
MaskCLIP* [53]	✗	32.9	16.3	22.9	25.5	17.5	61.8	14.2	25.0	27.0
GEM [3]	✗	46.8	-	34.5	-	-	-	17.1	-	-
SCLIP [42]	✗	59.1	30.5	30.4	34.2	22.4	80.4	16.1	32.2	38.2
ClearCLIP [25]	✗	51.8	33.0	32.6	35.9	23.9	80.9	16.7	30.0	38.1
CLIP-DIY† [45]	✓	59.9	31.0	19.7	19.8	13.3	79.7	9.9	11.6	30.6
CLIP-DINOiser* [46]	✓	62.2	34.7	32.5	36.0	24.6	80.8	20.1	31.1	40.2
ProxyCLIP* [26]	✓	59.3	**36.3**	34.4	38.0	25.7	79.7	19.4	36.0	41.1
LaVG* [23]	✓	61.8	33.3	31.5	34.6	22.8	**81.9**	14.8	25.0	38.2
LPOSS	✓	61.1	33.4	34.6	37.8	25.9	78.8	21.8	37.3	41.3
LPOSS+	✓	**62.4**	34.3	**35.4**	**38.6**	**26.5**	79.3	**22.3**	**37.9**	**42.1**

Table 1. **Performance comparison in terms of mIoU on 8 datasets** using ViT-B/16 backbone for both VLM and VM. * denotes the methods for which we reproduce the performance. † denotes numbers taken from Wysoczanska *et al*. [46].

Method	VM	VOC	Object	Context	C59	Stuff	VOC20	ADE20k	City	Avg
Oracle (patch-level res.)	-	80.1	61.3	62.6	68.0	69.0	83.5	65.0	66.4	69.5
MaskCLIP [53]	✗	17.2	7.0	9.2	12.0	9.3	38.6	8.2	17.2	14.8
CLIP-DINOiser [46]	✓	47.4	17.5	15.7	22.4	15.6	71.1	12.3	22.1	28.0
ProxyCLIP [26]	✓	45.0	19.7	17.4	23.0	15.9	69.4	12.9	28.4	29.0
LaVG [23]	✓	49.5	19.1	18.3	23.6	15.4	**74.3**	9.2	15.7	28.1
LPOSS	✓	48.5	18.4	19.1	24.8	17.9	70.1	14.9	28.4	30.3
LPOSS+	✓	**51.2**	**20.4**	**21.6**	**27.1**	**19.2**	71.7	**16.0**	**29.5**	**32.1**

Table 2. **Performance comparison in terms of Boundary IoU on 8 datasets** using ViT-B/16 backbone both for VLM and VM.

semantic segmentation. Following standard practice, we report mean Intersection over Union (mIoU) metric and additionally Boundary IoU [9] that measures performance only near the ground-truth mask boundaries.

4.2. Experimental setup

Backbones and textual prompts. We report the results using ViT-B/16 backbones and use DINO [5] as a visual model h and OpenCLIP [10] as a VLM (f, g). We follow the setup of CLIP-DINOiser [46] and use the output of DINO's last layer's *value embedding* as the visual model features Z_{vm}, while we use the output of MaskCLIP [53] as the VLM features Z_{vlm} and F. We use ImageNet prompt templates from CLIP [33] as templates for the VLM, following standard practice [23, 26, 46, 53]. For datasets that have a class `background`, we use text expansion for it and expand `background` into, *e.g.* `sky`, `wall`, *etc*., following SCLIP [42], ProxyCLIP [26], and LaVG [23].

Compared methods We compare our method with methods that use only a VLM, by making small changes during inference to the general ViT architecture, for open-vocabulary semantic segmentation: **MaskCLIP** [53], **GEM** [3], **SCLIP** [42], and **ClearCLIP** [25]. Additionally, we compare with methods that utilize vision models to aid VLMs: **CLIP-DIY** [45], **CLIP-DINOiser** [46][4], **ProxyCLIP** [26], and **LaVG** [23].

Implementation details We reproduce results for CLIP-DINOiser[5], ProxyCLIP[6], and LaVG[7] following their official implementations, while we reproduce MaskCLIP using the implementation provided by CLIP-DINOiser. We report the performance of CLIP-DIY [45] as reported in CLIP-DINOiser [46], while for GEM [3], SCLIP [42], and ClearCLIP [25] we report the numbers provided by the original papers. We implement our method using MMSegmentation [11] using the sliding-window approach with window size 224×224, window stride 112×112, and resizing the input image to have the shorter side of 448 for all datasets. We also fix the values of hyper-parameters α, k, γ, σ, r, and τ for LPOSS and LPOSS+ to 0.95, 400, 3.0, 100, 13, and 0.01, respectively, across all datasets. For pixel features of LPOSS+, we use image pixels converted to Lab color space. We discuss the design choices and show the impact of different hyper-parameters of LPOSS and LPOSS+ in Section 7 of the supplementary.

4.3. Results

We present the quantitative results in Table 1 given by the conventionally used mIoU metric. LPOSS achieves state-

[4]We use their trained model in our evaluation because it is publicly available and performs similarly to the training-free variant.

[5]https://github.com/wysoczanska/clip_dinoiser

[6]https://github.com/mc-lan/ProxyCLIP

[7]https://github.com/dahyun-kang/lavg

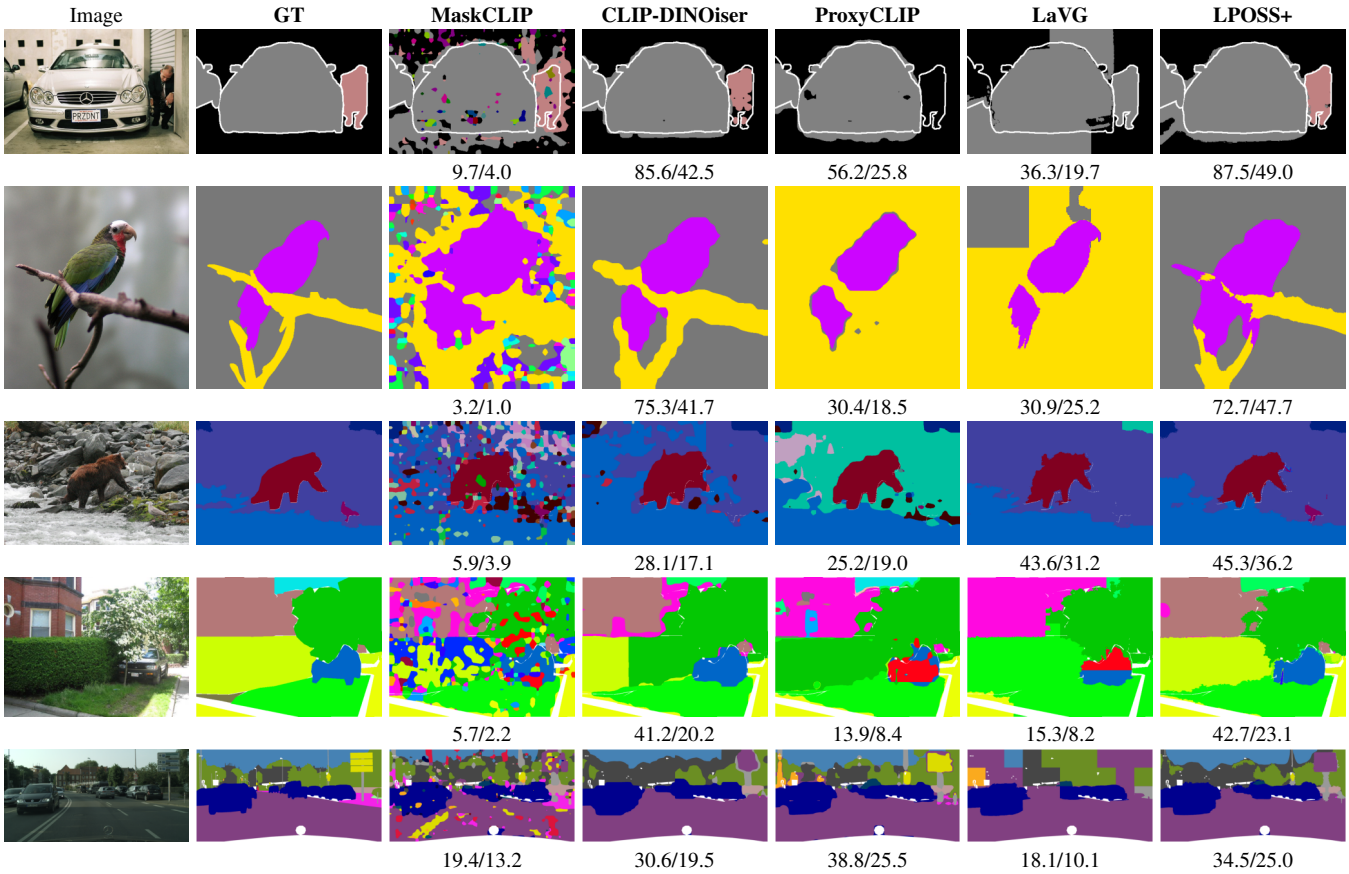


Figure 4. **Qualitative comparison of open-vocabulary semantic segmentation.** A comparison of LPOSS+ with the best performing methods: MaskCLIP [53], CLIP-DINOiser [46], ProxyCLIP [26], and LaVG [23]. On top of each segmentation map we show its mIoU/Boundary IoU. Pixels shown in white are pixels that do not have a class in the ground-truth.

of-the-art performance on average, with competitive performance across most datasets with only a few notable exceptions. One such exception happens on the VOC20 dataset where LPOSS has low performance compared to others. We observe this is the case because of the choice of the window size, something that we discuss in other experiments. LPOSS+ further improves the results of LPOSS by 0.8% on average and obtains state-of-the-art performance on all datasets besides VOC20 and Object. ProxyCLIP is the only method that significantly outperforms LPOSS+ on the Objects dataset, something that we attribute to the fact that it employs dataset-specific thresholds to filter the background class in this case, as well as for the VOC dataset.

Boundary IoU metric. Based on Table 1 we can already see that going beyond patch-level predictions in LPOSS to pixel-level predictions in LPOSS+ improves the results. Improvements are even more visible when we look into the Boundary IoU [9] metric, presented in Table 2, which is representative of how well the predicted object boundaries match the ground truth. LPOSS+ outperforms all other methods and achieves state-of-the-art on almost all datasets. Interestingly, even LPOSS outperforms competitors in this metric by a large margin. We attribute this to our choice of the window size, as LPOSS achieves Boundary IoU of 28.9% on average when we apply it with window size 448×448 and window stride 224×224, which is comparable to the other approaches.

Additionally, in Table 1 and Table 2, we report mIoU and Boundary IoU, respectively, for the oracle-based experiment for patch-level resolution prediction. In this experiment, we downsample the ground-truth map to the patch-level resolution by a factor equal to the patch size and upsample it to the original image resolution, which we treat as predictions. Both mIoU and especially Boundary IoU are significantly impacted by this process, which shows that patch-level predictions are significantly limiting the segmentation performance.

Qualitative results. We present the qualitative results of LPOSS+ and the most important competitors in Figure 4. We observe that LPOSS+ obtains good results across a va-

Method	VOC	Object	Context	C59	Stuff	VOC20	ADE20k	City	Avg
No sliding windows									
CLIP-DINOiser [46]	**60.2**	**32.4**	31.3	34.6	23.2	79.1	18.7	28.2	38.5
ProxyCLIP [26]	43.1	24.2	25.9	28.8	18.7	73.1	13.6	13.6	30.1
LaVG [23]	24.3	16.1	12.7	13.7	10.0	32.5	8.2	6.8	15.6
LPOSS	59.2	31.2	31.4	34.4	23.3	80.7	19.0	28.4	38.4
LPOSS+	59.7	31.7	**32.0**	**35.0**	**23.6**	**81.3**	**19.3**	**28.5**	**38.9**
Window size: 448×448*, window stride:* 224×224									
CLIP-DINOiser [46]	**62.2**	34.7	32.5	36.0	24.6	80.8	20.1	31.1	40.2
ProxyCLIP [26]	59.9	**36.0**	**34.6**	**38.2**	**25.6**	78.9	18.9	**33.6**	40.7
LaVG [23]	36.7	19.6	16.4	18.7	13.8	44.3	10.2	10.8	21.3
LPOSS	61.3	32.5	32.9	36.3	25.2	82.8	20.4	31.8	40.4
LPOSS+	62.0	33.1	33.4	36.9	25.5	**83.0**	**20.7**	32.1	**40.8**
Window size: 224×224*, window stride:* 112×112									
CLIP-DINOiser [46]	55.4	33.0	32.2	35.7	24.8	73.1	21.1	36.3	38.9
ProxyCLIP [26]	54.9	32.9	33.2	36.4	24.3	73.0	18.6	36.0	38.7
LaVG [23]	62.0	32.3	31.8	34.7	22.8	**79.8**	15.4	23.1	37.7
LPOSS	61.1	33.4	34.6	37.8	25.9	78.8	21.8	37.3	41.3
LPOSS+	**62.4**	**34.3**	**35.4**	**38.6**	**26.5**	79.3	**22.3**	**37.9**	**42.1**
Ensemble of two setups above									
CLIP-DINOiser [46]	63.0	**35.9**	34.1	37.8	26.3	79.6	22.1	35.9	41.8
ProxyCLIP [26]	59.0	35.1	35.3	38.9	26.1	78.1	19.8	36.8	41.1
LaVG [23]	51.4	27.3	24.3	28.4	20.1	63.7	14.7	20.3	31.3
LPOSS	62.9	34.1	35.1	38.6	26.5	82.1	22.2	36.5	42.3
LPOSS+	**63.9**	35.0	**35.8**	**39.3**	**27.0**	**82.5**	**22.7**	**37.0**	**42.9**

Table 3. **Impact of sliding windows for feature extraction.** Feature extraction is performed with windows of different size/stride or no windows at all. Following the original design choices, other methods perform window-based prediction, while LPOSS and LPOSS+ perform prediction jointly across all windows. Image resize of the smaller side to 448 is used and the reported metric is mIoU.

riety of images coming from different benchmark datasets. Qualitatively, our output is closer to that of CLIP-DINOiser due to the similarity of the approach. Nevertheless, ours varies in a smoother way, boosting the performance, which indicates that our LP-based formulation is a key ingredient. Additionally, in Figure 1, we show the qualitative comparison of LPOSS and LPOSS+. We observe that predictions of LPOSS+ follow the object boundary much better than the predictions from LPOSS.

Impact of sliding windows. In Table 3, we show the impact of sliding windows. All methods are negatively impacted when we depart from the use of sliding windows, with ProxyCLIP and LaVG being significantly more impacted than others. The largest drop in the performance happens on the Cityscapes dataset, which has an aspect ratio that is furthest from the square aspect ratio that was used to train VLM and VM. Besides that, CLIP-DINOiser and ProxyCLIP perform better when used with larger sliding windows, while LaVG, LPOSS, and LPOSS+ prefer smaller windows. We attribute LPOSS's and LPOSS+'s preference for smaller windows to the fact that we *decouple* prediction and feature extraction. We are thus able to select a (smaller) window size close to the training resolution to get stronger features, but are still able to propagate labels over the full image. Related methods lose performance in this setting because the prediction happens with the smaller context. Additionally, based on the analysis in Section 6 of the supplementary we propose to ensemble the two different window sizes. We observe that all methods, besides LaVG, benefit from the ensemble. However, LPOSS and LPOSS+ benefit more than others and obtain state-of-the-art results.

LPOSS++ as a refinement step for linear semantic segmentation. LPOSS+ refines pixel-level predictions and, as such, could potentially be applied to other dense prediction tasks. To test this, we apply it on the predictions obtained by training a linear segmentation head on top of DINOv2 [32] features on the ADE20k dataset [51, 52]. Applying LPOSS+ out of the box, we are able to improve DINOv2 predictions and increase mIoU from 47.3% to 49.5%.

5. Conclusion

This work proposes a training-free method for open-vocabulary segmentation, called LPOSS, that uses ViT-based VLMs and VMs. We highlight the limitations of low-resolution patch-based predictions and window-based processing. LPOSS targets both issues and provides improvements through an LP-based predictor that operates jointly over patches of all windows, and over pixels. In a comprehensive benchmark across a variety of datasets and domains, LPOSS surpasses the previous state-of-the-art, especially when evaluating near class boundaries.

Acknowledgements. This work was supported by the Junior Star GACR GM 21-28830M and the Czech Technical University in Prague grant No. SGS23/173/OHK3/3T/13. We acknowledge VSB – Technical University of Ostrava, IT4Innovations National Supercomputing Center, Czech Republic, for awarding this project access to the LUMI supercomputer, owned by the EuroHPC Joint Undertaking, hosted by CSC (Finland) and the LUMI consortium through the Ministry of Education, Youth and Sports of the Czech Republic through the e-INFRA CZ (grant ID: 90254).

References

[1] Nikita Araslanov and Stefan Roth. Single-stage semantic segmentation from image labels. In *CVPR*, 2020. 14

[2] Yoshua Bengio, Olivier Delalleau, and Nicolas Le Roux. Label propagation and quadratic criterion. In *Semi-Supervised Learning*, pages 192–216. The MIT Press, 2006. 3

[3] Walid Bousselham, Felix Petersen, Vittorio Ferrari, and Hilde Kuehne. Grounding everything: Emerging localization properties in vision-language transformers. In *CVPR*, 2024. 2, 6

[4] Holger Caesar, Jasper R. R. Uijlings, and Vittorio Ferrari. Coco-stuff: Thing and stuff classes in context. In *CVPR*, 2018. 5

[5] Mathilde Caron, Hugo Touvron, Ishan Misra, Hervé Jégou, Julien Mairal, Piotr Bojanowski, and Armand Joulin. Emerging properties in self-supervised vision transformers. In *ICCV*, 2021. 2, 3, 4, 6, 14

[6] Junbum Cha, Jonghwan Mun, and Byungseok Roh. Learning to generate text-grounded mask for open-world semantic segmentation from only image-text pairs. In *CVPR*, 2023. 2

[7] Siddhartha Chandra and Iasonas Kokkinos. Fast, exact and multi-scale inference for semantic image segmentation with deep gaussian crfs. In *ECCV*, 2016. 3

[8] Liang-Chieh Chen, George Papandreou, Iasonas Kokkinos, Kevin Murphy, and Alan L. Yuille. Deeplab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected crfs. *IEEE TPAMI*, 2018. 3

[9] Bowen Cheng, Ross Girshick, Piotr Dollár, Alexander C. Berg, and Alexander Kirillov. Boundary IoU: Improving object-centric image segmentation evaluation. In *CVPR*, 2021. 2, 5, 6, 7

[10] Mehdi Cherti, Romain Beaumont, Ross Wightman, Mitchell Wortsman, Gabriel Ilharco, Cade Gordon, Christoph Schuhmann, Ludwig Schmidt, and Jenia Jitsev. Reproducible scaling laws for contrastive language-image learning. In *CVPR*, 2023. 1, 2, 6

[11] MMSegmentation Contributors. MMSegmentation: Openmmlab semantic segmentation toolbox and benchmark. https://github.com/open-mmlab/mmsegmentation, 2020. 6

[12] Marius Cordts, Mohamed Omran, Sebastian Ramos, Timo Rehfeld, Markus Enzweiler, Rodrigo Benenson, Uwe Franke, Stefan Roth, and Bernt Schiele. The cityscapes dataset for semantic urban scene understanding. In *CVPR*, 2016. 5

[13] Timothée Darcet, Maxime Oquab, Julien Mairal, and Piotr Bojanowski. Vision transformers need registers. In *ICLR*, 2024. 2

[14] Jian Ding, Nan Xue, Gui-Song Xia, and Dengxin Dai. Decoupling zero-shot semantic segmentation. In *CVPR*, 2022. 2

[15] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021. 1, 3

[16] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman. The PASCAL Visual Object Classes Challenge 2012 (VOC2012) Results. 5

[17] Golnaz Ghiasi, Xiuye Gu, Yin Cui, and Tsung-Yi Lin. Scaling open-vocabulary image segmentation with image-level labels. In *ECCV*, 2022. 2

[18] Xuming He, Richard S. Zemel, and Miguel Á. Carreira-Perpiñán. Multiscale conditional random fields for image labeling. In *CVPR*, 2004. 3

[19] Xuefeng Hu, Ke Zhang, Lu Xia, Albert Chen, Jiajia Luo, Yuyin Sun, Ken Wang, Nan Qiao, Xiao Zeng, Min Sun, Cheng-Hao Kuo, and Ram Nevatia. Reclip: Refine contrastive language image pre-training with source free domain adaptation. In *WACV*, 2024. 3

[20] Ahmet Iscen, Giorgos Tolias, Yannis Avrithis, Teddy Furon, and Ondrej Chum. Efficient diffusion on region manifolds: Recovering small objects with compact CNN representations. In *CVPR*, 2017. 3

[21] Ahmet Iscen, Giorgos Tolias, Yannis Avrithis, and Ondrej Chum. Label propagation for deep semi-supervised learning. In *CVPR*, 2019. 3, 11

[22] Chao Jia, Yinfei Yang, Ye Xia, Yi-Ting Chen, Zarana Parekh, Hieu Pham, Quoc V. Le, Yun-Hsuan Sung, Zhen Li, and Tom Duerig. Scaling up visual and vision-language representation learning with noisy text supervision. In *ICML*, 2021. 1, 2

[23] Dahyun Kang and Minsu Cho. In defense of lazy visual grounding for open-vocabulary semantic segmentation. In *ECCV*, 2024. 2, 4, 5, 6, 7, 8, 12, 14, 15

[24] Philipp Krähenbühl and Vladlen Koltun. Efficient inference in fully connected crfs with gaussian edge potentials. In *NeurIPS*, 2011. 3, 14

[25] Mengcheng Lan, Chaofeng Chen, Yiping Ke, Xinjiang Wang, Litong Feng, and Wayne Zhang. Clearclip: Decomposing clip representations for dense vision-language inference. In *ECCV*, 2024. 2, 6

[26] Mengcheng Lan, Chaofeng Chen, Yiping Ke, Xinjiang Wang, Litong Feng, and Wayne Zhang. Proxyclip: Proxy attention improves clip for open-vocabulary segmentation. In *ECCV*, 2024. 1, 2, 4, 5, 6, 7, 8, 12, 14, 15

[27] Yi Li, Hualiang Wang, Yiqun Duan, Hang Xu, and Xiaomeng Li. Exploring visual interpretability for contrastive language-image pre-training. *arXiv preprint arXiv:2209.07046*, 2022. 2

[28] Feng Liang, Bichen Wu, Xiaoliang Dai, Kunpeng Li, Yinan Zhao, Hang Zhang, Peizhao Zhang, Peter Vajda, and Diana

Marculescu. Open-vocabulary semantic segmentation with mask-adapted CLIP. In *CVPR*, 2023. 2

[29] Huaishao Luo, Junwei Bao, Youzheng Wu, Xiaodong He, and Tianrui Li. Segclip: Patch aggregation with learnable centers for open-vocabulary semantic segmentation. In *ICML*, 2023. 2

[30] Roozbeh Mottaghi, Xianjie Chen, Xiaobai Liu, Nam-Gyu Cho, Seong-Whan Lee, Sanja Fidler, Raquel Urtasun, and Alan L. Yuille. The role of context for object detection and semantic segmentation in the wild. In *CVPR*, 2014. 5

[31] Jishnu Mukhoti, Tsung-Yu Lin, Omid Poursaeed, Rui Wang, Ashish Shah, Philip H. S. Torr, and Ser-Nam Lim. Open vocabulary semantic segmentation with patch aligned contrastive learning. In *CVPR*, 2023. 2

[32] Maxime Oquab, Timothée Darcet, Theo Moutakanni, Huy V. Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Russell Howes, Po-Yao Huang, Hu Xu, Vasu Sharma, Shang-Wen Li, Wojciech Galuba, Mike Rabbat, Mido Assran, Nicolas Ballas, Gabriel Synnaeve, Ishan Misra, Herve Jegou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. DINOv2: Learning robust visual features without supervision. *TMLR*, 2024. 2, 8

[33] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *ICML*, 2021. 1, 2, 3, 6

[34] Kanchana Ranasinghe, Brandon McKinzie, Sachin Ravi, Yinfei Yang, Alexander Toshev, and Jonathon Shlens. Perceptual grouping in contrastive vision-language models. In *ICCV*, 2023. 2

[35] Jamie Shotton, John M. Winn, Carsten Rother, and Antonio Criminisi. *TextonBoost*: Joint appearance, shape and context modeling for multi-class object recognition and segmentation. In *ECCV*, 2006. 3

[36] Oriane Siméoni, Gilles Puy, Huy V. Vo, Simon Roburin, Spyros Gidaris, Andrei Bursuc, Patrick Pérez, Renaud Marlet, and Jean Ponce. Localizing objects with self-supervised transformers and no labels. In *BMVC*, 2021. 2, 4

[37] Oriane Siméoni, Chloé Sekkat, Gilles Puy, Antonín Vobecký, Éloi Zablocki, and Patrick Pérez. Unsupervised object localization: Observing the background to discover objects. In *CVPR*, 2023. 2, 4

[38] Vladan Stojnić, Yannis Kalantidis, and Giorgos Tolias. Label propagation for zero-shot classification with vision-language models. In *CVPR*, 2024. 2, 3, 11

[39] Boyuan Sun, Yuqi Yang, Le Zhang, Ming-Ming Cheng, and Qibin Hou. Corrmatch: Label propagation via correlation matching for semi-supervised semantic segmentation. In *CVPR*, 2024. 3

[40] Charles Sutton and Andrew McCallum. An introduction to conditional random fields. *Found. Trends Mach. Learn.*, 2012. 3

[41] Vishaal Udandarao, Ankush Gupta, and Samuel Albanie. Sus-x: Training-free name-only transfer of vision-language models. In *ICCV*, 2023. 2

[42] Feng Wang, Jieru Mei, and Alan Yuille. Sclip: Rethinking self-attention for dense vision-language inference. In *ECCV*, 2024. 1, 2, 4, 5, 6

[43] Jianzong Wu, Xiangtai Li, Shilin Xu, Haobo Yuan, Henghui Ding, Yibo Yang, Xia Li, Jiangning Zhang, Yunhai Tong, Xudong Jiang, Bernard Ghanem, and Dacheng Tao. Towards open vocabulary learning: A survey. *IEEE Trans. Pattern Anal. Mach. Intell.*, 46(7):5092–5113, 2024. 2

[44] Size Wu, Wenwei Zhang, Sheng Jin, Wentao Liu, and Chen Change Loy. Aligning bag of regions for open-vocabulary object detection. In *CVPR*, 2023. 2

[45] Monika Wysoczanska, Michaël Ramamonjisoa, Tomasz Trzcinski, and Oriane Siméoni. CLIP-DIY: CLIP dense inference yields open-vocabulary semantic segmentation for-free. In *WACV*, 2024. 2, 6

[46] Monika Wysoczanska, Oriane Siméoni, Michaël Ramamonjisoa, Andrei Bursuc, Tomasz Trzcinski, and Patrick Pérez. Clip-dinoiser: Teaching clip a few dino tricks for open-vocabulary semantic segmentation. In *ECCV*, 2024. 1, 2, 3, 4, 5, 6, 7, 8, 12, 14, 15

[47] Jiarui Xu, Shalini De Mello, Sifei Liu, Wonmin Byeon, Thomas M. Breuel, Jan Kautz, and Xiaolong Wang. Groupvit: Semantic segmentation emerges from text supervision. In *CVPR*, 2022. 2

[48] Jilan Xu, Junlin Hou, Yuejie Zhang, Rui Feng, Yi Wang, Yu Qiao, and Weidi Xie. Learning open-vocabulary semantic segmentation models from natural language supervision. In *CVPR*, 2023. 2

[49] Lihe Yang, Bingyi Kang, Zilong Huang, Zhen Zhao, Xiaogang Xu, Jiashi Feng, and Hengshuang Zhao. Depth anything V2. *arXiv preprint arXiv:2406.09414*, 2024. 12

[50] Shuai Zheng, Sadeep Jayasumana, Bernardino Romera-Paredes, Vibhav Vineet, Zhizhong Su, Dalong Du, Chang Huang, and Philip H. S. Torr. Conditional random fields as recurrent neural networks. In *ICCV*, 2015. 3

[51] Bolei Zhou, Hang Zhao, Xavier Puig, Sanja Fidler, Adela Barriuso, and Antonio Torralba. Scene parsing through ADE20K dataset. In *CVPR*, 2017. 5, 8

[52] Bolei Zhou, Hang Zhao, Xavier Puig, Tete Xiao, Sanja Fidler, Adela Barriuso, and Antonio Torralba. Semantic understanding of scenes through the ADE20K dataset. *IJCV*, 2019. 5, 8

[53] Chong Zhou, Chen Change Loy, and Bo Dai. Extract free dense labels from CLIP. In *ECCV*, 2022. 2, 4, 6, 7, 15

[54] Dengyong Zhou, Olivier Bousquet, Thomas Navin Lal, Jason Weston, and Bernhard Schölkopf. Learning with local and global consistency. In *NeurIPS*, 2003. 2, 3